Radix Sort(Linear Time Sorting)

10 May 2026 20:03

Introduction to Linear time sorting

Most comparison-based sorting algorithms like:

- Bubble sort
- Selection sort
- Insertion sort
- Merge sort
- Quick sort
- Heap sort

Have a lower bound of : O(n log n)
Because they depend on comparison between elements.
However, some special sorting algorithms can work in linear time:
O(n)
Under certain conditions.
These are called linear time sorting algorithms.
Examples:
a. Counting Sort
b. Radix sort
c. Bucket sort

What is Radix Sort?

Definition: Radix sort is a non-comparative sorting algorithm that sorts numbers digit by digit.
It processes digits from:

- Least significant digit (LSD)-> Most common.

Or

- Most significant digit (MSD)

Usually in Java, we use:

LSD Radix Sort(starting from units place)

Core Idea of Radix Sort

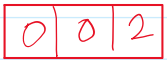
Suppose we have:
170, 45, 75, 90, 802, 24, 2, 66

Radix sort sorts numbers:
i. By unit digit
ii. Then tens digit
iii. The hundreds digit
After processing all digits, the array becomes sorted.

Important Property
Radix sort uses a stable sorting algorithm internally.
Usually:

Counting Sort: is used for each digit.

Working of Radix Sort



Consider:
170, 45, 75, 90, 802, 24, 2, 66

Step-1: Sort by unit digit: 170, 90, 802, 2, 24, 45, 75, 66

Step-2: Sort by tens digit: 802, 2, 24, 45, 66, 170, 75, 90

Step-3: Sort by hundred's digit: 2, 24, 45, 66, 75, 90, 170, 802

Algorithm of radix sort

1. Find maximum number
2. Count digits in maximum number
3. Apply Counting Sort for each digit
4. Move from LSD to MSD.

Java Code:

```
package LinearTimeSorting;
import java.util.Random;
public class RadixSort {
    // Function to get maximum value
    public static int getMax(int arr[]){
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if(arr[i] > max){
                max = arr[i];
            }
        }
        return max;
    }
    // Counting sort according to digit representation by exp
    public static void countingSort(int arr[], int exp){
        int n = arr.length;
        int output[] = new int[n];
        int count[] = new int[10];
        //Store count of occurrences
        for (int i = 0; i < n; i++) {
            int digit = (arr[i]/exp) % 10;
            count[digit]++;
        }
        //Change count[i] so that it contains actual position of
        //digit in output
        for (int i = 1; i < 10; i++) {
            count[i] += count[i-1];
        }
        //Build output array
```

```
        // Traverse from right to maintain stability
        for (int i = n - 1; i >=0 ; i--) {
            int digit =(arr[i]/exp) % 10;
            output[count[digit] - 1] = arr[i];
            count[digit]--;
        }
        // Copy output array to arr
        for (int i = 0; i < n; i++) {
            arr[i] = output[i];
        }
    }
    // Main radix sort function.
    public static void radixSort(int arr[]){
        int max = getMax(arr);
        // Apply counting sort for every digit
        for(int exp = 1; max/exp > 0; exp*=10){
            countingSort(arr, exp);
        }
    }
    //display array
    static void printArray(int arr[], String msg){
        System.out.println(msg);
        for(int x: arr){
            System.out.print(x + " ");
        }
        System.out.println();
    }
    //main function
    public static void main(String[] args) {
        Random r = new Random();
        int A[] = new int[r.nextInt(10, 21)];
        for (int i = 0; i < A.length; i++) {
            A[i] = r.nextInt(1, 100);
        }
        printArray(A, "Before sorting: ");
        radixSort(A);
        printArray(A, "After sorting: ");
    }
}
```

Homework: Do the dry of the code.